SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction

COURSE NAME: Data Structures **COURSE CODE:** CSA03

Register Number	**192572129**
Name	**Pooja Shree K N**

Assessment Tool Weightage: 35%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: 5 **EACH QUESTIONS CARRIES:** 5 MARKS **Total Marks:** 25 **Duration:** ONE HOUR

S. No.	List of Questions	

1	**Array Traversal Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[5] = {10, 20, 30, 40, 50};`
		`  int i;`
		`  for(i = 0; i < 5; i++)`
		`  printf("%d ", a[i]);`
		`  return 0;`
		`}`

2	**Sum of Array Elements Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[] = {2, 4, 6, 8};`
		`  int sum = 0, i;`
		
		`  for(i = 0; i < 4; i++)`
		`  sum += a[i];`
		
		`  printf("%d", sum);`
		`  return 0;`
		`}`

3	**Recursive Factorial Program**	`#include <stdio.h>`
		
		`int fact(int n)`
		`{`
		`  if(n == 0)`
		`  return 1;`
		`  return n * fact(n - 1);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fact(4));`
		`  return 0;`
		`}`

4	**Recursive Fibonacci Program**	`#include <stdio.h>`
		
		`int fib(int n)`
		`{`
		`  if(n <= 1)`
		`  return n;`
		`  return fib(n-1) + fib(n-2);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fib(6));`
		`  return 0;`
		`}`

5	**Function as ADT Operation Program**	`#include <stdio.h>`
		
		`int add(int x, int y)`
		`{`
		`  return x + y;`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", add(15, 25));`
		`  return 0;`
		`}`

Code of Conduct:

I Pooja Shree K N(192572129) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Pooja Shree K N

RUBRICS FOR CALCULATION:

Criteria	Weightage (5 Marks)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (3 Marks)	Level 1 – Beginning (1–2 Marks)
Understanding of Program Logic	1	Correctly identifies the complete program flow and execution sequence.	Minor misunderstanding of program flow.	Partial understanding of logic.	Unable to interpret the program logic.
Trace of Variables / Function Calls	1	Accurately traces all variable values, iterations, and recursive calls.	Traces most values correctly with minor errors.	Traces some values but misses important steps.	Incorrect or no tracing.
Output Prediction Accuracy	2	Output predicted completely and correctly.	Output mostly correct with minor mistakes.	Output partially correct.	Output incorrect or missing.
Complexity Interpretation (Time/Space)	1	Correctly identifies and justifies complexity where applicable.	Correct complexity with limited justification.	Partial understanding of complexity.	Incorrect or no complexity analysis.

1. Array Traversal Program

```c
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;
    for(i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Output

```
10 20 30 40 50
```

Concepts used:

Array declaration and initialization

Array traversal using a for loop

Array indexing (a[i])

`printf()` to display elements

Logic: The loop runs from i = 0 to i = 4 and prints each array element.

Complexity: Time — \(O(n)\); Auxiliary space — \(O(1)\).

2. Sum of Array Elements Program

```c
#include <stdio.h>
int main()
{
    int a[] = {2, 4, 6, 8};
    int sum = 0, i;

    for(i = 0; i < 4; i++)
        sum += a[i];

    printf("%d", sum);
    return 0;
}
```

Output

Concepts used:

Array declaration and initialization

Looping through array elements

Accumulator variable (`sum`)

Addition using `sum += a[i]`

Logic: The program adds all elements: \(2 + 4 + 6 + 8 = 20\).

Complexity: Time — \(O(n)\); Auxiliary space — \(O(1)\).

3 . Recursive Factorial Program

```c
#include <stdio.h>

int fact(int n)
{
    if(n == 0)
        return 1;
    return n * fact(n - 1);
}

int main()
{
    printf("%d", fact(4));
    return 0;
}
```

Output

24

Concepts used:

Recursion

User-defined function (fact())

Base condition (if (n == 0))

Recursive function calls

Multiplication

Logic: The factorial of 4 is:

\[4! = 4 \times 3 \times 2 \times 1 = 24 \]

Function calls: fact(4) ⟶ 4 × fact(3) ⟶ 4 × 3 × fact(2) ⟶ 4 × 3 × 2 × fact(1) ⟶ 4 × 3 × 2 × 1 × fact(0).

Complexity: Time — \(O(n)\); Auxiliary space — \(O(n)\), due to recursive calls.

4 . **Recursive Fibonacci**
Program

```c
#include <stdio.h>

int fib(int n)
{
    if(n <= 1)
        return n;
    return fib(n-1) + fib(n-2);
}

int main()
{
    printf("%d", fib(6));
    return 0;
}
```

Output

8

Concepts used:

Recursion

Fibonacci sequence

Base condition (if (n <= 1))

Multiple recursive calls

Addition

Logic: The function calculates fib(6) using:

\[fib(n) = fib(n-1) + fib(n-2) \]

The sequence is 0, 1, 1, 2, 3, 5, 8. Therefore, fib(6) = 8.

Complexity: Time — \(O(2^n)\); Auxiliary space — \(O(n)\).

5.Function as ADT Operation Program

```c
#include <stdio.h>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    printf("%d", add(15, 25));
    return 0;
}
```

Output

40

Concepts used:

User-defined function

Function arguments and parameters

Function call

Return statement

Abstraction (function hides the implementation of the addition operation)

Logic: The function add(x, y) receives 15 and 25, adds them, and returns 40.

$$add(15,25) = 15 + 25 = 40$$

Complexity: Time — $O(1)$; Auxiliary space — $O(1)$.